

A IDP PROJECT REPORT
On
"Healthcare Medical chatbot"

Submitted

By

231FA04231 - M Fanindhar
231FA04567 - K Venkata Lokesh
231FA04D11 - A.Hemanth Sai

Under the guidance of

Ms. V. Sai Spandana

Assistant Professor



SCHOOL OF COMPUTING & INFORMATICS
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
VIGNAN'S FOUNDATION FOR SCIENCE, TECHNOLOGY AND RESEARCH
Deemed to be UNIVERSITY
Vadlamudi, Guntur.
ANDHRA PRADESH, INDIA, PIN-522213.



VIGNAN'S

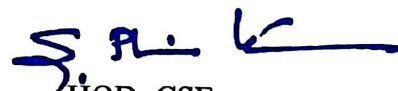
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

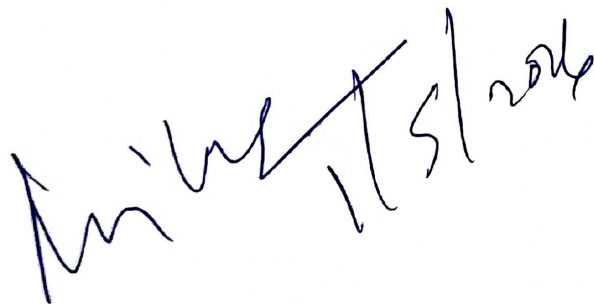
CERTIFICATE

This is to certify that the IDP Project entitled “**Healthcare Medical chatbot**” that is being submitted by 231FA04231 (M Fanindhar), 231FA04567 (K Venkata Lokesh) and 231FA04D11 (A.Hemanth Sai) in partial fulfilment of the requirements for the IDP Project, as a part of the academic curriculum of the B.Tech (CSE) Program, Department of Computer Science and Engineering at VFSTR Deemed to be University.


Project Guide


HOD, CSE

HoD
Dept. of Computer Science & Engineering
VFSTR Deemed to be University
VADLAMUDI - 522-213
Guntur Dist., A.P., India.





VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

DECLARATION

We hereby declare that the IDP Project entitled "**Healthcare Medical chatbot**" that is being submitted by 231FA04231(M.Fanindhar), 231FA04567(K.Venkata Lokesh) and 231FA04D11(A.Hemanth Sai) in partial fulfilment of IDP Project course work. This is our original work, and it has not been submitted previously for the award of any degree or diploma. The project has been completed under the supervision of Ms. V. Sai Spandana, Assistant Professor, Department of Computer Science and Engineering, VFSTR Deemed to be University.

By
231FA04231(M Fanindhar),
231FA04567(K Venkata Lokesh),
231FA04D11 (A.Hemanth Sai)

ABSTARCT

The “Healthcare Medical Chatbot” project is an AI-powered web application designed to provide preliminary medical assistance and healthcare guidance to users through an intelligent chatbot system. The main objective of the project is to reduce the burden on healthcare services by offering 24/7 automated support for symptom analysis, disease prediction, and doctor recommendations. The system enables patients to interact with the chatbot using natural language, where the entered symptoms are processed using Natural Language Processing (NLP) and Machine Learning techniques.

The application is developed using React.js for the frontend interface, Flask for backend API services, and MySQL for database management. A fine-tuned BERT model is integrated into the system to improve the accuracy of symptom understanding and disease prediction. The chatbot analyzes patient symptoms, predicts possible diseases with confidence scores, suggests specialist doctors, and allows users to book appointments directly through the platform.

The system contains three major modules: Patient, Doctor, and Administrator. Patients can register, log in, interact with the chatbot, view diagnosis results, and book appointments. Doctors can manage appointments, access patient records, and update consultation notes. Administrators are responsible for user management, monitoring system activities, and updating the medical knowledge base. The platform also ensures security and privacy through authentication mechanisms, encrypted data transmission, and secure cloud deployment on AWS infrastructure.

The Healthcare Medical Chatbot improves accessibility to healthcare services, especially for users in remote areas and during non-clinic hours. The project demonstrates the effective integration of Artificial Intelligence, NLP, cloud computing, and full-stack web technologies to build an intelligent healthcare assistance platform. The developed system is scalable, user-friendly, and capable of future enhancements such as multilingual support, telemedicine integration, and IoT-based health monitoring.

TABLE OF CONTENTS

LIST OF FIGURES	7
LIST OF TABLES	8
ABBREVIATIONS	9
1. INTRODUCTION	13
1.1 Introduction	13
1.2 Project Background	14
1.3 Objective	14
1.4 Project Description	14
2. SOFTWARE REQUIREMENTS SPECIFICATION	17
2.1 Requirement Analysis	17
2.2 Problem Statement	17
2.3 Functional Requirements	17
2.4 Software Requirement Specification	18
2.5 Software Requirements	20
2.6 Hardware Requirements	21
2.7 Functional Requirements (Modules)	21
2.8 Non-Functional Requirements	23
2.9 External Interface Requirements	23
2.10 Feasibility Study	24
3. ANALYSIS & DESIGN	27
3.1 Introduction	27
3.2 System Overview	28
3.3 System Architecture	29
3.4 Data Design	34
4. MODELING	39
4.1 Design	39
5. IMPLEMENTATION	46
5.1 Sample Code	46
5.2 Screen Captures	55
6. TESTING	62
6.1 Software Testing	62
6.2 Black Box Testing	62
6.3 White Box Testing	62
6.4 Performance Testing	62
6.5 Load Testing	62
6.6 Manual Testing	62
7. RESULTS AND CHALLENGES	66
7.1 Results	66
7.2 Challenges	66

8. CONCLUSION.....	68
8.1 Conclusions	68
8.2 Scope for Future Work.....	68
8.3 Limitations	68
BIBLIOGRAPHY	6

LIST OF FIGURES

Figure 3-1 System Architecture of Healthcare Medical Chatbot	28
Figure 3-2 Web Application Architecture Diagram	30
Figure 3-3 NLP Pipeline Architecture	32
Figure 4-1 Use Case Diagram for System	38
Figure 4-2 Sequence Diagram for Patient Module	39
Figure 4-3 Sequence Diagram for Doctor Module	40
Figure 4-4 Activity Diagram for Medical Chatbot	40
Figure 4-5 Class Diagram for Medical Chatbot	41
Figure 4-6 Deployment Diagram of the System	42
Figure 4-7 ER Diagram	43
Figure 5-1 Home Screen	54
Figure 5-2 Patient Registration Screen	55
Figure 5-3 Patient Login Screen	55
Figure 5-4 Chatbot Conversation Screen	56
Figure 5-5 Symptom Input Activity	56
Figure 5-6 Disease Prediction Result	56
Figure 5-7 Doctor Recommendation	57
Figure 6-1 Test Case for Symptom Input	62

LIST OF TABLES

Table 2.1 Client Requirements	20
Table 2.2 Server Requirements	20
Table 3.1 MySQL Database	33 Table
3.2 List of Database Tables	33
Table 3.3 Structure of Users Table	33 Table
3.4 Structure of Symptoms Table	34 Table
3.5 Structure of Diseases Table	35
Table 3.6 Structure of Appointments Table	35

ABBREVIATIONS

AI : ARTIFICIAL INTELLIGENCE

ML : MACHINE LEARNING

NLP : NATURAL LANGUAGE PROCESSING

API : APPLICATION PROGRAMMING INTERFACE

UI : USER INTERFACE

UX : USER EXPERIENCE

JSON : JAVASCRIPT OBJECT NOTATION

REST : REPRESENTATIONAL STATE TRANSFER

HTTP : HYPERTEXT TRANSFER PROTOCOL

HTML : HYPER TEXT MARKUP LANGUAGE

CSS : CASCADING STYLE SHEETS

JS : JAVASCRIPT

SDK : SOFTWARE DEVELOPMENT KIT

IDE : INTEGRATED DEVELOPMENT ENVIRONMENT

DB : DATABASE

SQL : STRUCTURED QUERY LANGUAGE

AWS : AMAZON WEB SERVICES

EMR : ELECTRONIC MEDICAL RECORDS

NLTK : NATURAL LANGUAGE TOOLKIT

BERT : BIDIRECTIONAL ENCODER REPRESENTATIONS FROM TRANSFORMERS

CHAPTER - 1

INTRODUCTION

1. INTRODUCTION

1.1 Introduction

The current project involves the development of an AI-powered Healthcare Medical Chatbot that assists patients in understanding their symptoms, obtaining preliminary diagnoses, and connecting with appropriate healthcare professionals. The main aim of the project is to reduce the burden on healthcare systems by providing 24/7 automated medical guidance and information to users.

In this web application, users can interact with the chatbot by entering their symptoms in natural language. The chatbot processes the input using Natural Language Processing (NLP) techniques and responds with relevant medical information, possible conditions, and recommendations to consult a specialist. The system also allows users to book appointments with doctors available on the platform.

In this project, we are going to create a platform where there are three kinds of users namely Patients, Doctors, and Administrators.

1. Patient:

This user can register on the platform and log in to interact with the chatbot. The patient can describe their symptoms in natural language, receive AI-driven health suggestions, view recommended doctors nearby, and book appointments.

2. Doctor:

This user can sign in to the platform and manage their availability, view scheduled appointments, access patient interaction history, and update prescriptions and consultation notes.

3. Administrator:

This user manages the overall platform including user registrations, doctor approvals, disease database updates, and system monitoring.

We will be using Flask as the backend framework, React.js for the frontend, and the platform will be deployed on Amazon Web Services (AWS) cloud infrastructure.

1.2 Project Background

The healthcare industry faces significant challenges in providing timely and accessible medical assistance to all patients. Long waiting times, shortage of doctors in rural areas, and lack of health awareness among the general population are persistent problems. With the rapid advancement of Artificial Intelligence and Natural Language Processing, it is now feasible to build intelligent systems that can simulate a preliminary consultation with a medical professional.

This project was conceived to address these gaps by creating a chatbot that not only understands patient symptoms but also provides structured health guidance based on a curated medical knowledge base. The system integrates machine learning models trained on medical datasets to achieve accurate symptom-to-disease mapping.

1.3 Objective

The objective of the Healthcare Medical Chatbot system is to:

- Automate the preliminary diagnosis process using AI/ML models trained on symptom-disease datasets.
- Provide patients with 24/7 access to health information and guidance without requiring a physical visit.
- Maintain a detailed profile of each patient including medical history, past consultations, and prescribed medicines.
- Facilitate appointment booking between patients and registered doctors on the platform.
- Allow doctors to manage consultations and patient records digitally through a dedicated dashboard.
- Ensure data security and patient privacy through proper authentication and encryption mechanisms.

1.4 Project Description

In this section, all features of the application are explained in brief.

1) Patient

i) Register / Sign In

ii) Log In

(a) Interact with Medical Chatbot

(b) View Diagnosis Results

(c) Book Appointment

(d) View Appointment History

iii) Log Out

a) Register:

A new patient can join the platform by providing personal details including name, date of birth, gender, phone number, email address, and a secure password. After registration, a unique patient ID is assigned.

b) Login:

The patient must enter their registered email address and password to access the platform. A 'Forgot Password' option is available to reset credentials via registered email.

i. Interact with Chatbot:

After logging in, the patient can start a conversation with the chatbot by entering symptoms in natural language. The chatbot responds with a list of possible conditions along with their severity levels and brief descriptions.

ii. View Diagnosis Results:

The patient can view AI-generated diagnostic suggestions based on the entered symptoms. The results include possible diseases, recommended specialist type, and urgency level.

iii. Book Appointment:

Based on the recommended specialist, the patient can browse available doctors and book an appointment at a convenient date and time.

2) Doctor

- i) Register / Sign In
- ii) Log In
 - (a) View Appointments
 - (b) Manage Patient Records
 - (c) Update Consultation Notes
- iii) Log Out

3) Administrator

- i) Log In
- ii) Manage Users (Patients & Doctors)
- iii) Update Medical Knowledge Base
- iv) Monitor System Health
- v) Log Out

CHAPTER - 2

SOFTWARE REQUIREMENTS SPECIFICATION

2. SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Requirement Analysis

For the purpose of easy access and portability, we proposed to develop a web-based system that is compatible with all major operating systems including Windows, macOS, and Linux. The system is designed to run on any device with a modern web browser. As part of this system, we are going to develop web-based software in which data can be accessed and retrieved efficiently. The required documents for these processes are as follows:

1. Problem Statement
2. Data Flow Diagrams
3. Use Case Diagram
4. Other UML Diagrams

The above-mentioned documents provide a diagrammatical view of the system we are going to develop.

2.2 Problem Statement

The problem statement concentrates on the inaccessibility of timely medical guidance to patients, especially in rural areas and during non-clinic hours. The solution involves building an intelligent healthcare chatbot backed by a MySQL database for storing user profiles, medical history, symptom data, and appointment records, synced with a secure cloud server.

2.3 Functional Requirements

Patient Registration and Login:

1. The platform shows registration and login options. If a patient is not registered, they can sign up.
2. The platform collects the patient's first name and last name.
3. The platform collects the patient's date of birth.
4. The platform collects the patient's gender.
5. The platform collects the patient's phone number.
6. The platform collects the patient's email address.
7. The platform requires the patient to set and confirm a password.
8. The platform collects the patient's city/location for nearby doctor recommendations.

2. Chatbot Interaction:

9. After login, the patient can begin a conversation with the chatbot.
10. The patient enters symptoms in natural language text.
11. The chatbot responds with possible diseases, severity levels, and recommendations.

3. Appointment Booking:

12. The patient can view available doctors based on the recommended specialty.
13. The patient can select a doctor and choose a time slot to book an appointment.

2.4 Software Requirement Specification

The project is developed using Python (Flask) as the backend framework and React.js for the frontend. The NLP engine uses the BERT model fine-tuned on medical symptom datasets. MySQL is used as the relational database and all services are deployed on the AWS cloud platform.

2.4.1 Purpose

The purpose of this document is to present a detailed description of the 'Healthcare Medical Chatbot' application. It will explain the purpose and features of the system, the constraints under which it must operate, and how the system will react to various inputs. The document also describes the non-functional requirements of the system.

2.4.2 Scope of the Project

In this project, we are going to create a web application consisting of three kinds of users: Patients, Doctors, and Administrators.

1. Patient:

This user will register with personal details and can subsequently log in to interact with the AI chatbot, receive diagnostic suggestions, view doctor recommendations, and schedule appointments.

2. Doctor:

This user will register with professional credentials (subject to admin approval) and can log in to view and manage patient appointments, update consultation records, and communicate with patients through the platform.

3. Administrator:

This user manages the entire platform including user approvals, medical knowledge base updates, system health monitoring, and report generation.

2.4.3 Technologies Used

PYTHON (FLASK):

Flask is a lightweight WSGI web application framework written in Python. It provides the tools, libraries, and technologies needed to build a web application. Flask follows the WSGI standard and is designed with simplicity and flexibility in mind. It is used here as the backend REST API server.

REACT.JS:

React is a free and open-source front-end JavaScript library for building user interfaces based on components. It is maintained by Meta and a community of developers and companies. React can be used to develop single-page, mobile, or server-rendered applications with frameworks like Next.js.

NATURAL LANGUAGE PROCESSING (NLP) - BERT:

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based machine learning technique for natural language processing. The chatbot uses a fine-tuned BERT model to understand patient-reported symptoms and map them to possible medical conditions.

HTML & CSS:

Hypertext Markup Language and Cascading Style Sheets are used for structuring and styling the web pages of the application, providing a responsive and user-friendly interface.

JAVASCRIPT:

JavaScript is used for client-side interactivity, form validation, dynamic chatbot conversation rendering, and API communication with the Flask backend.

MYSQL:

MySQL is one of the most popular relational database management systems on the web. It is used here to store and manage patient profiles, doctor records, appointment data, medical history, and disease datasets with high speed and security.

REST API:

The communication between the React frontend and the Flask backend is implemented using RESTful APIs, enabling a clean separation of concerns and scalable architecture.

2.4.4 Overview

The application is a full-stack web-based system that communicates with a cloud server to send and retrieve medical data as required. The system works with internet connectivity and uses HTTPS for all data transmissions to ensure security and privacy of patient data.

2.5 Software Requirements

The software interface includes the operating system and application programming interfaces used for development:

- Operating System: Windows 10 / Ubuntu 20.04 LTS
- Backend Framework: Python Flask 2.x
- Frontend Framework: React.js 18.x
- Database: MySQL 8.0
- NLP Model: BERT (via Hugging Face Transformers)
- Cloud Platform: Amazon Web Services (AWS EC2 + RDS)
- Technologies: Python, JavaScript, HTML5, CSS3, REST API

CLIENT	OPERATING SYSTEM	SOFTWARE	DISK SPACE	RAM
Any PC/Mobile	Any OS	Any Web Browser	Min 100 MB	512 MB

Table 2.1 Client Requirements

SERVER	OPERATING SYSTEM	SOFTWARE	PROCESSOR	RAM	DISK SPACE
AWS EC2	Ubuntu 20.04 LTS	Flask 2.x, MySQL 8.0	Intel Xeon 2.4GHz	4 GB	Min 20 GB

Table 2.2 Server Requirements

2.7 Functional Requirements (Modules)

The project has three main modules: Patient, Doctor, and Administrator, each with definite functionality.

1. Patient Module:

- i) Register
- ii) Login
 - (a) Chat with Medical Chatbot
 - (b) View Diagnosis & Recommendations
 - (c) Book Appointment
 - (d) View Appointment History
- iii) Logout

2. Doctor Module:

- i) Register (with professional verification)
- ii) Login
 - (a) View Scheduled Appointments
 - (b) Access Patient Medical History
 - (c) Update Consultation Notes & Prescriptions
- iii) Logout

3. Administrator Module:

- i) Login
- ii) Manage Patient & Doctor Accounts
- iii) Update Disease & Symptom Knowledge Base
- iv) View System Reports & Analytics
- v) Logout

2.8 Non-Functional Requirements

2.8.1 Performance Requirements

Performance requirements define acceptable response times for system functionality. The page load time for user interface screens shall take no longer than 5 seconds. The login authentication shall be verified within 3 seconds. Chatbot responses shall be generated and returned within 10 seconds under normal load conditions.

2.8.2 Design Constraints

The Healthcare Medical Chatbot shall be a web-based system accessible from any browser. The system shall be developed using React.js for the frontend and Flask for the backend, connected to a MySQL database on AWS RDS. The chatbot interface must be responsive for both desktop and mobile devices.

2.8.3 Standards Compliance

There shall be consistency in variable names, API endpoint naming conventions (RESTful standards), and UI component design within the system. The graphical user interface shall have a consistent look and feel across all pages.

2.8.4 Availability

The system shall be available 24 hours a day, 7 days a week, as the primary use case involves patients seeking medical information at any time of day.

2.8.5 Portability

The Healthcare Medical Chatbot shall run in any environment that contains a modern web browser with internet connectivity. It shall be accessible on Windows, macOS, Linux, Android, and iOS platforms without requiring any additional software installation.

2.8.6 Reliability

Since the application is developed using Python Flask and React.js, two of the most widely used and well-supported frameworks, the system is reliable in all standard use cases. Proper error handling, input validation, and fallback mechanisms are implemented to maintain system stability.

2.8.7 Security

All patient data is encrypted at rest and in transit using industry-standard encryption (AES-256, HTTPS/TLS). User authentication is implemented using JWT (JSON Web Tokens) with expiry mechanisms. Password hashing uses the bcrypt algorithm.

2.9 External Interface Requirements

2.9.1 User Interface

A critical aspect of this project was designing an intuitive and accessible user interface. The frontend is built using React.js with a clean conversational chat interface for the chatbot interaction. The application is responsive and supports both desktop and mobile browsers.

When the application is launched, it displays the home page with options to register as a new user or log in as an existing patient, doctor, or administrator. The chatbot interface mimics a messaging application for a familiar user experience.

2.9.2 Database Schema Overview

Users Table: Name, Phone Number, Email, Date of Birth, Gender, Location, Role (Patient/Doctor/Admin), Password Hash

Symptoms Table: Symptom ID, Symptom Name, Symptom Description, Related Disease IDs Diseases

Table: Disease ID, Disease Name, Description, Severity Level, Recommended Specialist

Appointments Table: Appointment ID, Patient ID, Doctor ID, Date, Time Slot, Status, Notes

2.10 Feasibility Study

A key part of the preliminary investigation reviews anticipated costs and benefits, and recommends a course of action based on operational, technical, economic, and time factors.

2.10.1 Organisational Feasibility

The Healthcare Medical Chatbot would contribute to the overall objectives of digital health transformation. It provides a quick, accurate, and cost-effective solution to preliminary medical consultation. The system is flexible and scalable, capable of being upgraded with additional disease datasets, specialist domains, or multilingual support in the future.

2.10.2 Economic Feasibility

The project is economically feasible as it uses open-source frameworks (Flask, React.js, NLTK, Hugging Face). The primary cost is cloud hosting on AWS, which is minimal for a university-level deployment. The application is accessible via browser at no cost to end users.

2.10.3 Technical Feasibility

To develop this application, a cloud server (AWS EC2), a database server (AWS RDS/MySQL), an NLP engine (BERT via Hugging Face), and a web server (Nginx with Gunicorn) are required. The system has been successfully tested on AWS infrastructure and is technically feasible.

2.10.4 Behavioural Feasibility

The application is behaviourally feasible since it requires no technical knowledge from end users. The conversational chatbot interface is intuitive, and all modules are designed to be user-friendly. The AI responses are presented in simple, easy-to-understand language.

CHAPTER - 3

ANALYSIS & DESIGN

3. ANALYSIS & DESIGN

3.1 Introduction

3.1.1 Purpose

In this section, the purpose of the document and the project is described.

3.1.1.1 Document Purpose

A Software Design Document (SDD) is a representation of a software system used as a medium for communicating software design information. This document describes the architectural design, module interactions, and data design of the Healthcare Medical Chatbot system.

3.1.1.2 Project Purpose

The prime purpose of this 'Healthcare Medical Chatbot' application is to create a fully functional web application that communicates with a remote server to send and retrieve medical data as per patient requirements. This application works over internet connectivity and leverages AI/NLP to provide accurate, timely health information.

3.1.2 Scope

3.1.2.1 Document Scope

This document contains a thorough description of the high-level architecture used in developing the system. It conveys the overall system design, user interface design, and higher-level module design including NLP integration, RESTful API communication, and database schema design.

3.1.2.2 Project Scope

The application mainly consists of three modules: Patient, Doctor, and Administrator. The Patient module allows users to interact with the chatbot, receive health guidance, and book appointments. The Doctor module provides tools for managing appointments and patient records. The Administrator module handles system management and knowledge base updates.

3.2 System Overview

3.2.1.1 Visual Studio Code

Visual Studio Code is a lightweight but powerful source code editor available for Windows, macOS, and Linux. It provides built-in support for JavaScript, TypeScript, Python, and Node.js, with a rich ecosystem of extensions for React development, Flask debugging, and database management.

3.2.1.2 Python & Flask

Python is a high-level, general-purpose programming language emphasizing readability and simplicity. Flask is a micro web framework written in Python, used to build the REST API backend of this application. Flask supports extensions for authentication (Flask-JWT), database ORM (Flask-SQLAlchemy), and CORS handling.

3.2.1.3 Hugging Face Transformers

Hugging Face Transformers is an open-source library providing thousands of pre-trained models for NLP tasks. In this project, a pre-trained BERT model is fine-tuned on a medical symptom-disease dataset for the chatbot's diagnostic functionality. The library provides simple APIs for model loading, tokenization, and inference.

3.3 System Architecture

3.3.1 Architectural Design

The Healthcare Medical Chatbot follows a three-tier web application architecture consisting of the presentation layer (React.js frontend), the application logic layer (Flask REST API with NLP engine), and the data layer (MySQL database on AWS RDS).

Attributes of a well-built Healthcare Chatbot Architecture:

- Accurate and context-aware medical responses
- Secure patient data handling with encryption
- Scalable NLP model integration
- Fast and reliable API response times
- Ensures HIPAA-compliant data storage practices
- Supports concurrent users without performance degradation
- Provides comprehensive logging and audit trails
- Guarantees high availability through AWS auto-scaling

Components of the Healthcare Chatbot Architecture:

The system architecture consists of three primary layers:

- **Presentation Layer:** React.js SPA with responsive chatbot UI, patient dashboard, doctor dashboard, and admin panel.
- **Application Layer:** Flask REST API server handling authentication, chatbot inference, appointment management, and data retrieval.
- **Data Layer:** MySQL database storing all application data with AWS RDS for managed database services.

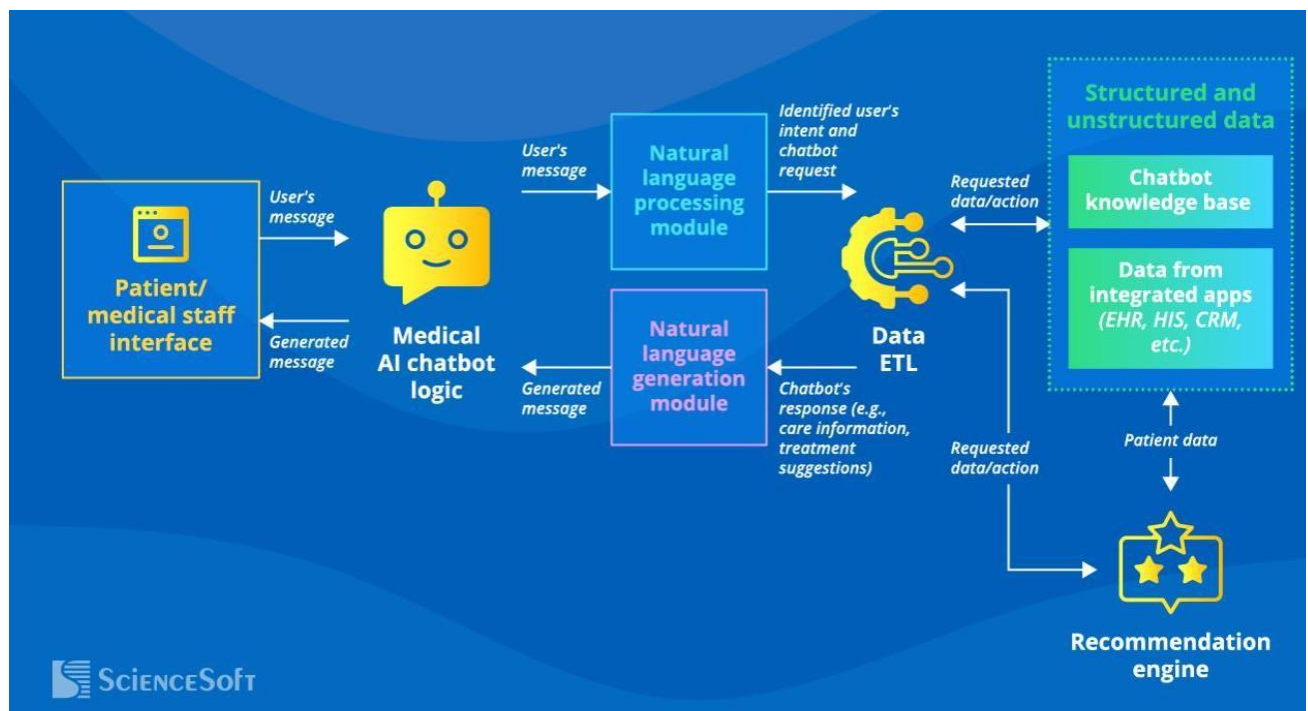


Figure 3-1 System Architecture of Healthcare Medical Chatbot

3.3.2 NLP and AI Engine

The core of the Healthcare Medical Chatbot is its NLP engine. When a patient enters a symptom description, the text is preprocessed, tokenized using the BERT tokenizer, and passed through the fine-tuned classification model. The model outputs a probability distribution over known diseases, and the top predictions are returned to the patient with confidence scores.

The NLP pipeline consists of:

- Text Preprocessing: Lowercasing, punctuation removal, and stop-word filtering.
- Tokenization: BERT tokenizer with WordPiece vocabulary.
- Feature Extraction: 768-dimensional embeddings from BERT's transformer layers.
- Classification: Fully connected layer with softmax output over disease classes.

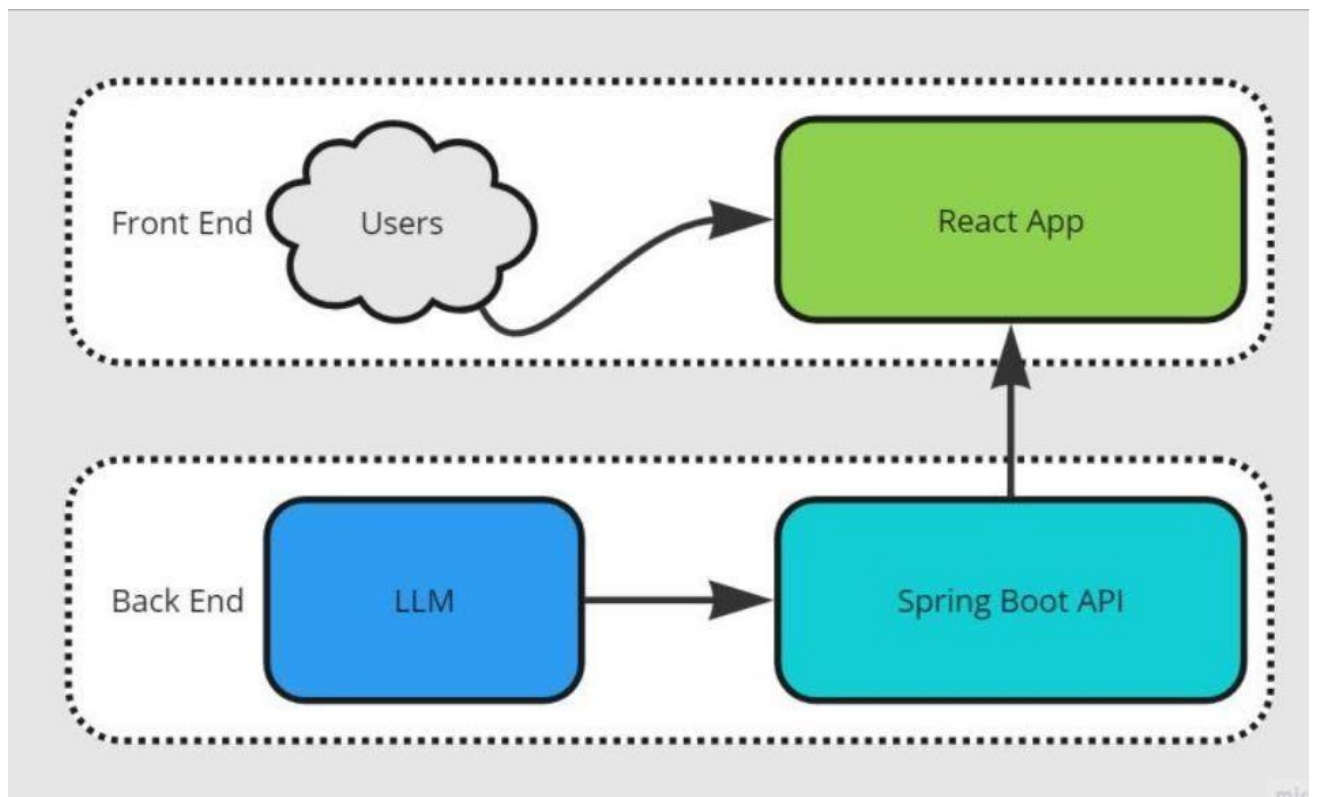


Figure 3-2 Web Application Architecture Diagram

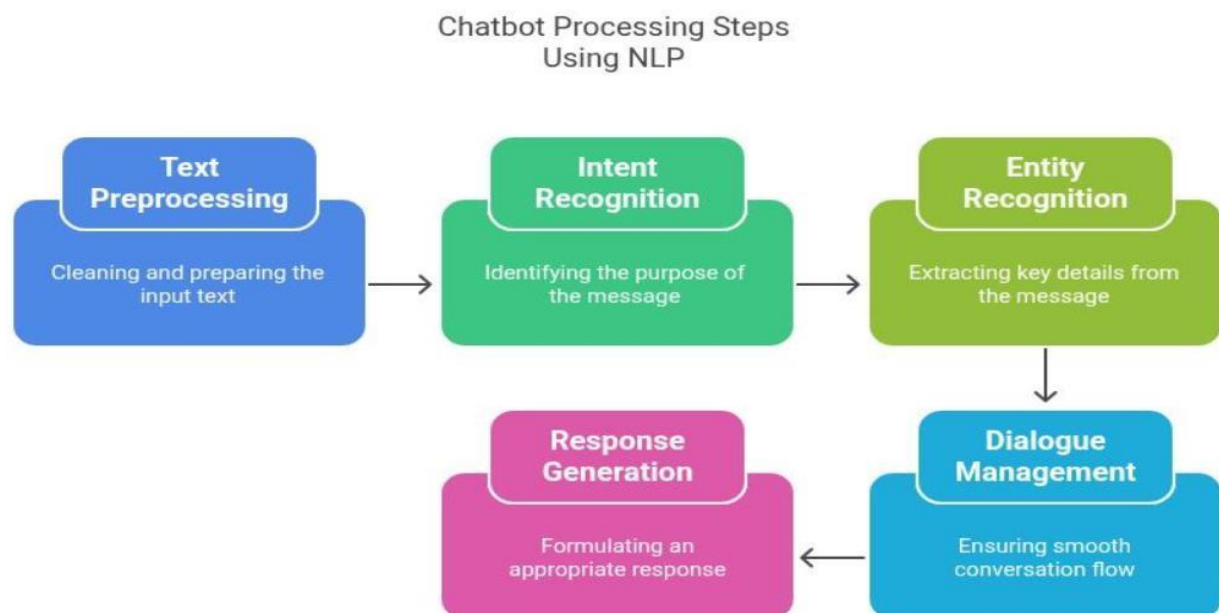


Figure 3-3 NLP Pipeline Architecture

3.3.3 Application Components

The Healthcare Medical Chatbot consists of the following major application components:

Presentation Layer (React.js):

The frontend is built as a Single Page Application (SPA) using React.js. It provides the chatbot conversation interface, patient registration and login forms, doctor and admin dashboards, appointment booking UI, and medical report viewer.

Business Logic Layer (Flask API):

The Flask backend handles all business logic including user authentication using JWT, chatbot response generation by invoking the NLP model, appointment scheduling, and data retrieval from the MySQL database. All endpoints follow RESTful conventions.

Data Access Layer:

Flask-SQLAlchemy ORM is used as the data access layer, abstracting direct SQL queries. The ORM models correspond to the database tables for Users, Symptoms, Diseases, Appointments, and Medical History.

NLP/AI Engine:

The BERT-based NLP engine is loaded as a module within the Flask application. Inference requests from the API are processed by the model, and the results are serialized to JSON and returned to the frontend.

3.3.4 Overall Software Architecture

The overall architecture follows the Model-View-Controller (MVC) pattern. The React.js frontend represents the View layer, the Flask API represents the Controller layer, and the MySQL database with SQLAlchemy models represents the Model layer. The BERT NLP engine is integrated as a service within the Controller layer.

Data flows from the patient's browser through HTTPS to the Flask API, which processes it via the NLP engine and retrieves/stores data in the MySQL database. All responses are returned as JSON and rendered dynamically in the React.js frontend.

3.4 Data Design

1.1.1 Databases

Name
healthcare_cms

Table 3.1 MySQL Database

users	To store patient, doctor, and admin details
symptoms	To store symptom definitions and mappings
diseases	To store disease information and severity levels
appointments	To store appointment booking records

Table 3.2 List of Database Tables

3.4.2.1 Database: healthcare_cms Table: users

user_id	INT	No	Auto Increment	Primary Key
name	VARCHAR(50)	No	NULL	-
email	VARCHAR(100)	No	NULL	Unique Key
phone	BIGINT	No	NULL	Unique Key
dob	DATE	No	NULL	-
gender	VARCHAR(10)	No	NULL	-
role	VARCHAR(20)	No	patient	-
password_hash	VARCHAR(256)	No	NULL	-
location	VARCHAR(100)	Yes	NULL	-

Table 3.3 Structure of users table of MySQL Database

3.4.2.2 Database: healthcare_cms Table: symptoms

Name	Type	Null	Default	Index
symptom_id	INT	No	Auto Increment	Primary Key
symptom_name	VARCHAR(100)	No	NULL	-
description	TEXT	Yes	NULL	-

Table 3.4 Structure of symptoms table of MySQL Database

3.4.2.3 Database: healthcare_cms Table: diseases

Name	Type	Null	Default	Index
disease_id	INT	No	Auto Increment	Primary Key
disease_name	VARCHAR(100)	No	NULL	-
description	TEXT	No	NULL	-
severity	VARCHAR(20)	No	NULL	-
specialist	VARCHAR(50)	No	NULL	-

Table 3.5 Structure of diseases table of MySQL Database

3.4.2.4 Database: healthcare_cms Table: appointments

Name	Type	Null	Default	Index
appointment_id	INT	No	Auto Increment	Primary Key
patient_id	INT	No	NULL	Foreign Key
doctor_id	INT	No	NULL	Foreign Key
appointment_date	DATE	No	NULL	-
time_slot	VARCHAR(20)	No	NULL	-
status	VARCHAR(30)	Yes	Scheduled	-
notes	TEXT	Yes	NULL	-

Table 3.6 Structure of appointments table of MySQL Database

CHAPTER - 4

MODELING

4. MODELING

4.1 Design

Requirements gathering followed by careful analysis leads to a systematic Object-Oriented Design (OOAD). Various activities have been identified and are represented using Unified Modeling Language (UML) diagrams. UML is used to specify, visualize, modify, construct, and document the artifacts of an object-oriented software-intensive system under development.

4.1.1. Use Case Diagram

In the Unified Modeling Language (UML), the use case diagram is a type of behavioral diagram defined by and created from a use-case analysis. It represents a graphical overview of the functionality of the system in terms of actors — which are persons, organizations, or external systems that play a role in one or more interactions with the system. These are drawn as stick figures. The goals of these actors are represented as use cases, which describe a sequence of actions that provide measurable value to an actor.

In this application, there are three actors: Patient, Doctor, and Administrator. The use case diagram below illustrates all major system interactions.

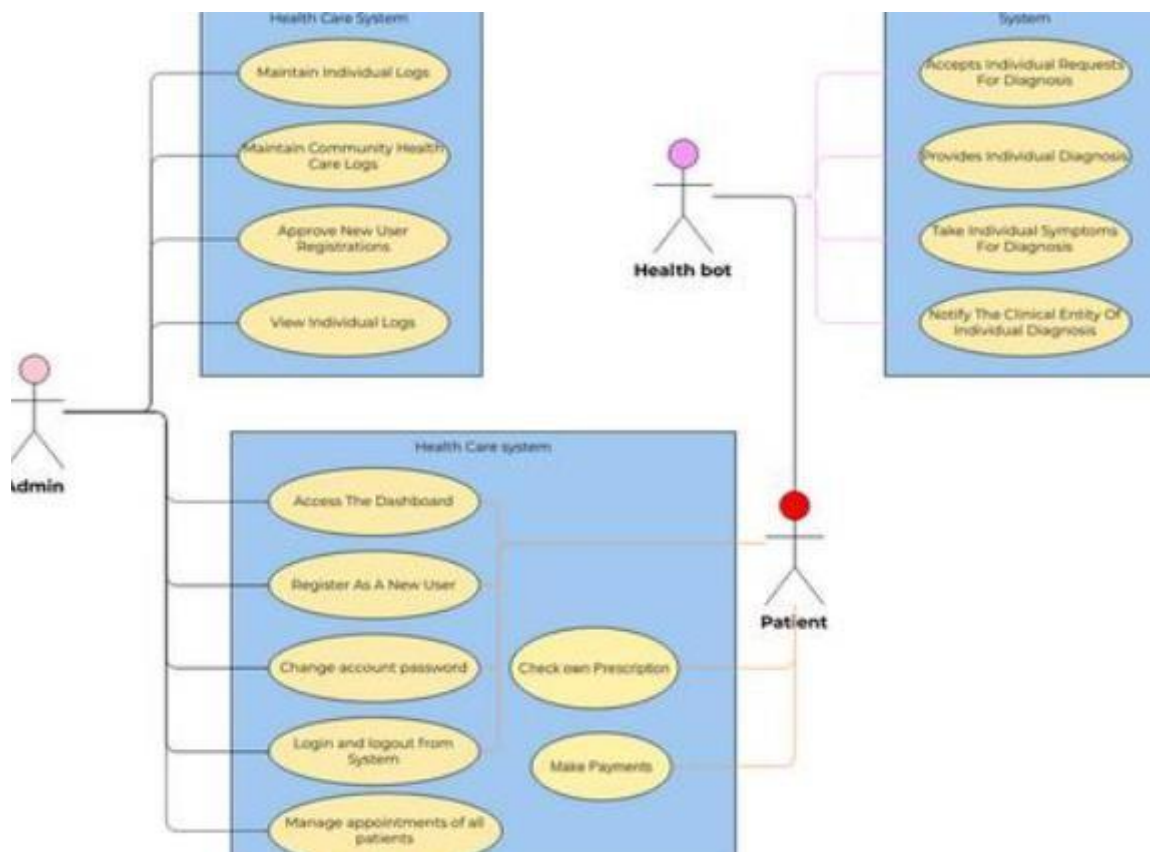


Figure 4-1 Use Case Diagram for System

4.1.2 Sequence Diagram

UML sequence diagrams are used to show how objects interact in a given situation. An important characteristic of a sequence diagram is that time passes from top to bottom: the interaction starts near the top of the diagram and ends at the bottom (i.e., lower equals later).

A popular use for them is to document the dynamics in an object-oriented system. For each key collaboration, diagrams are created that show how objects interact in various representative scenarios.

The sequence diagram for the Patient module shows the interaction between the patient, the React frontend, the Flask API, the BERT NLP engine, and the MySQL database during a chatbot consultation session.

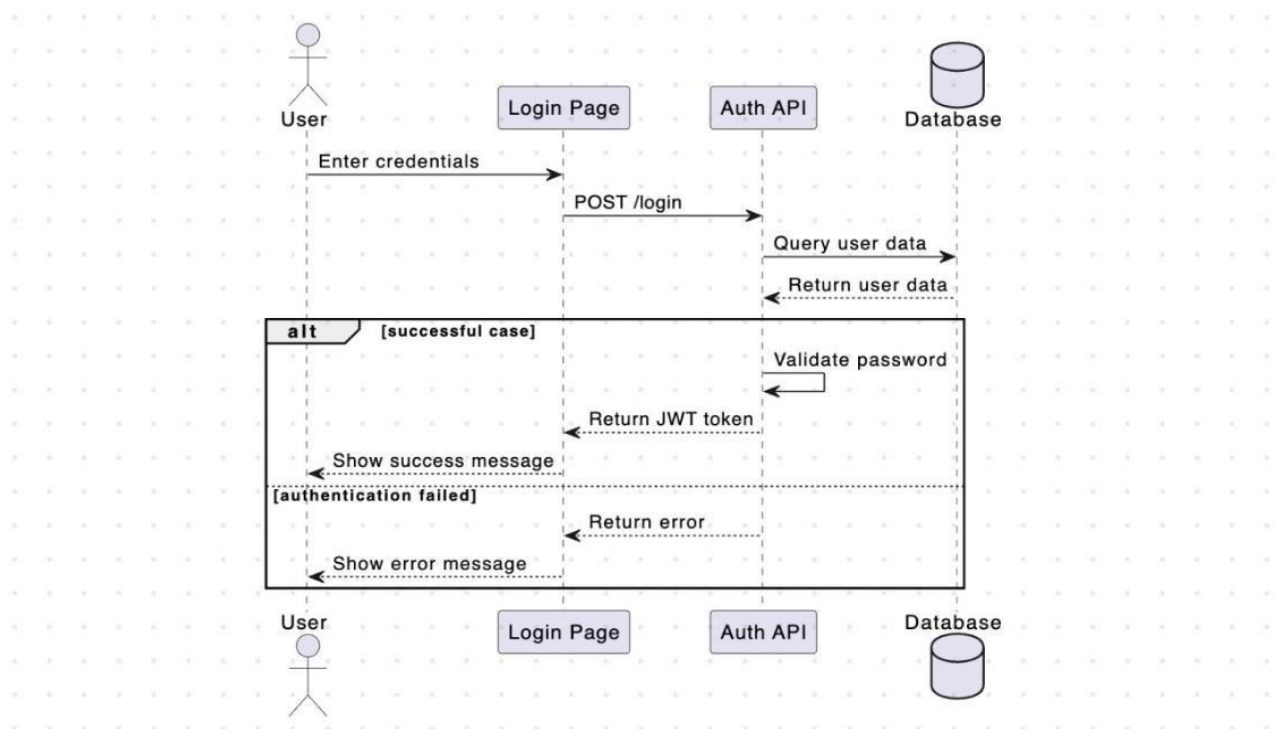


Figure 4-2 Sequence Diagram for Patient Module

Sequence Diagram of Online Doctor Appointment System:

A sequence diagram is a Unified Modeling Language (UML) diagram that illustrates the sequence of messages between objects in an interaction. A Sequence diagram consists of a group of objects that are represented by lifelines, and the messages that they exchange over time during the interaction.

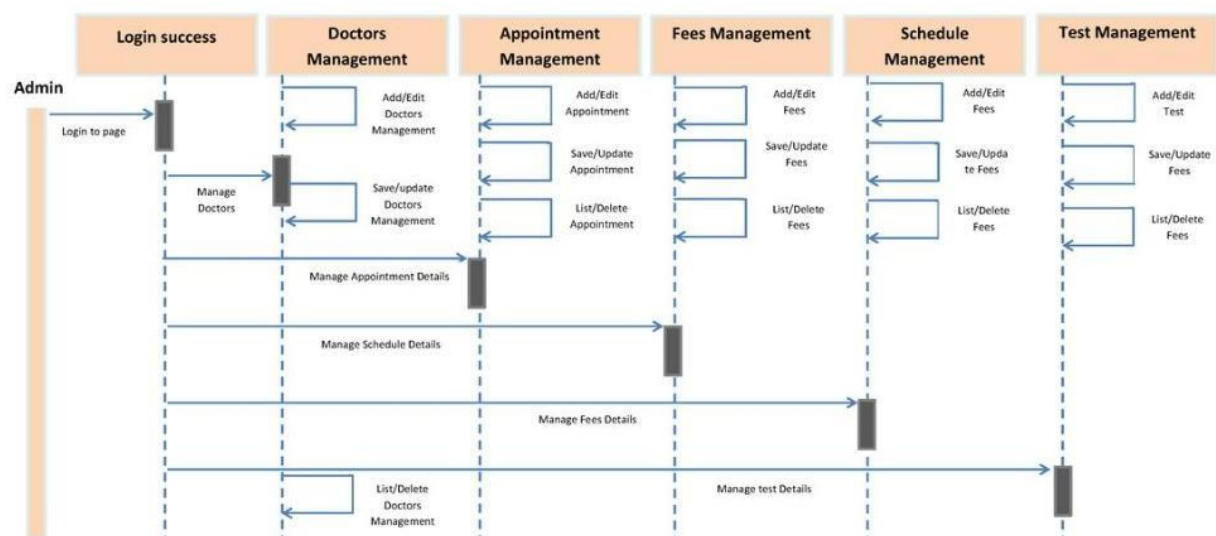


Figure 4-3 Sequence Diagram for Doctor Module

4.1.3 Activity Diagram

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. An activity diagram is basically a flowchart to represent the flow from one activity to another. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all types of flow control by using different elements like fork, join, and decision nodes.

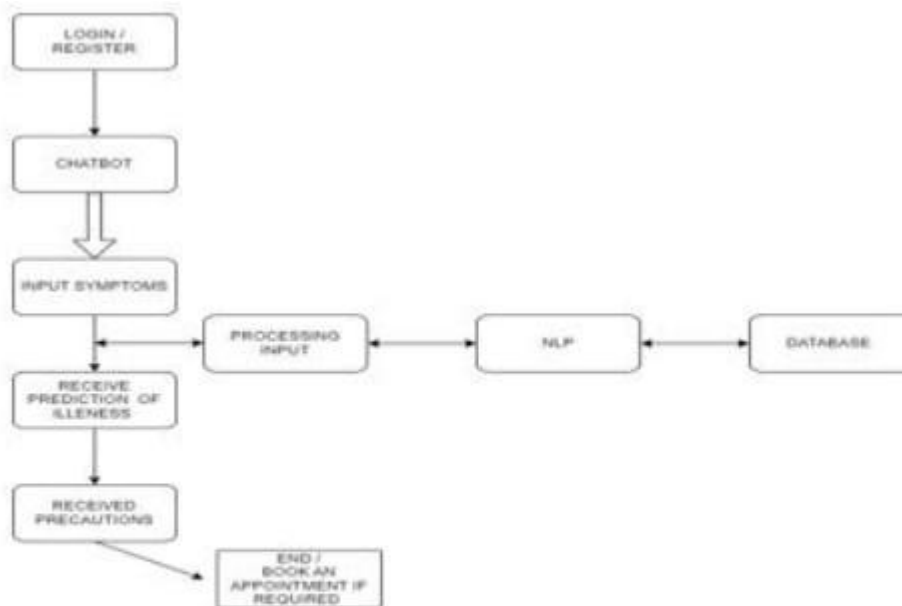


Fig 1: Interaction of Chatbot

Figure 4-4 Activity Diagram for Healthcare Medical Chatbot

4.1.4 Class Diagram

In software engineering, a class diagram in UML is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes.

The class diagram is the main building block of object-oriented modelling. It is used both for general conceptual modelling of the application and for detailed modelling translating the models into programming code. The primary classes in this application are: User, Patient, Doctor, Administrator, Chatbot, Disease, Symptom, Appointment, and MedicalHistory.

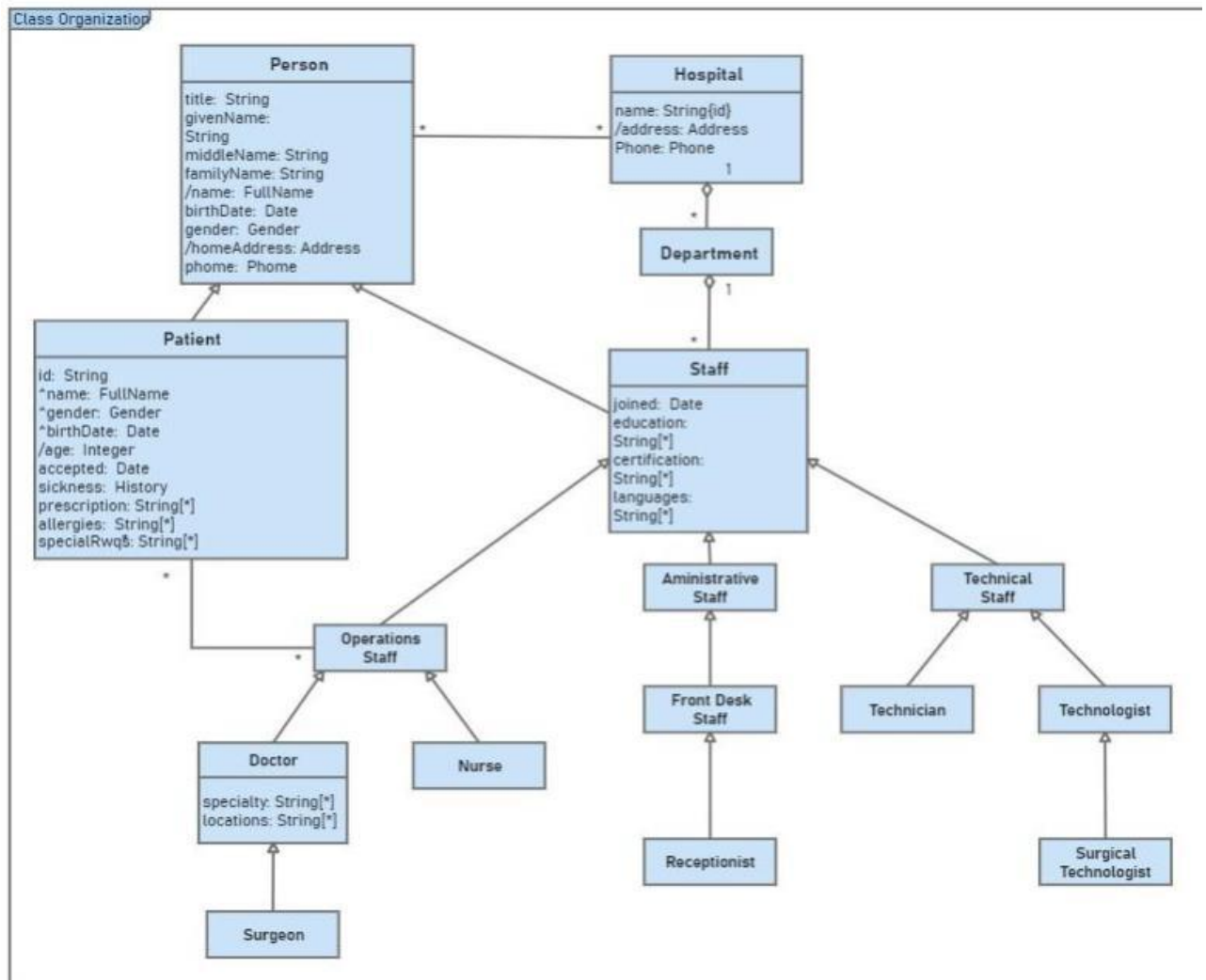


Figure 4-5 Class Diagram for Healthcare Medical Chatbot

4.1.5 Deployment Diagram

The deployment diagram shows the execution architecture of the system that represents the assignment (deployment) of software artifacts to deployment targets (usually nodes). Nodes represent either hardware devices or software execution environments.

The deployment diagram for this application shows: the user's browser (PC/Mobile), the React.js frontend hosted on AWS S3/CloudFront, the Flask API server on AWS EC2, the BERT NLP model served via a separate inference server, and the MySQL database on AWS RDS.

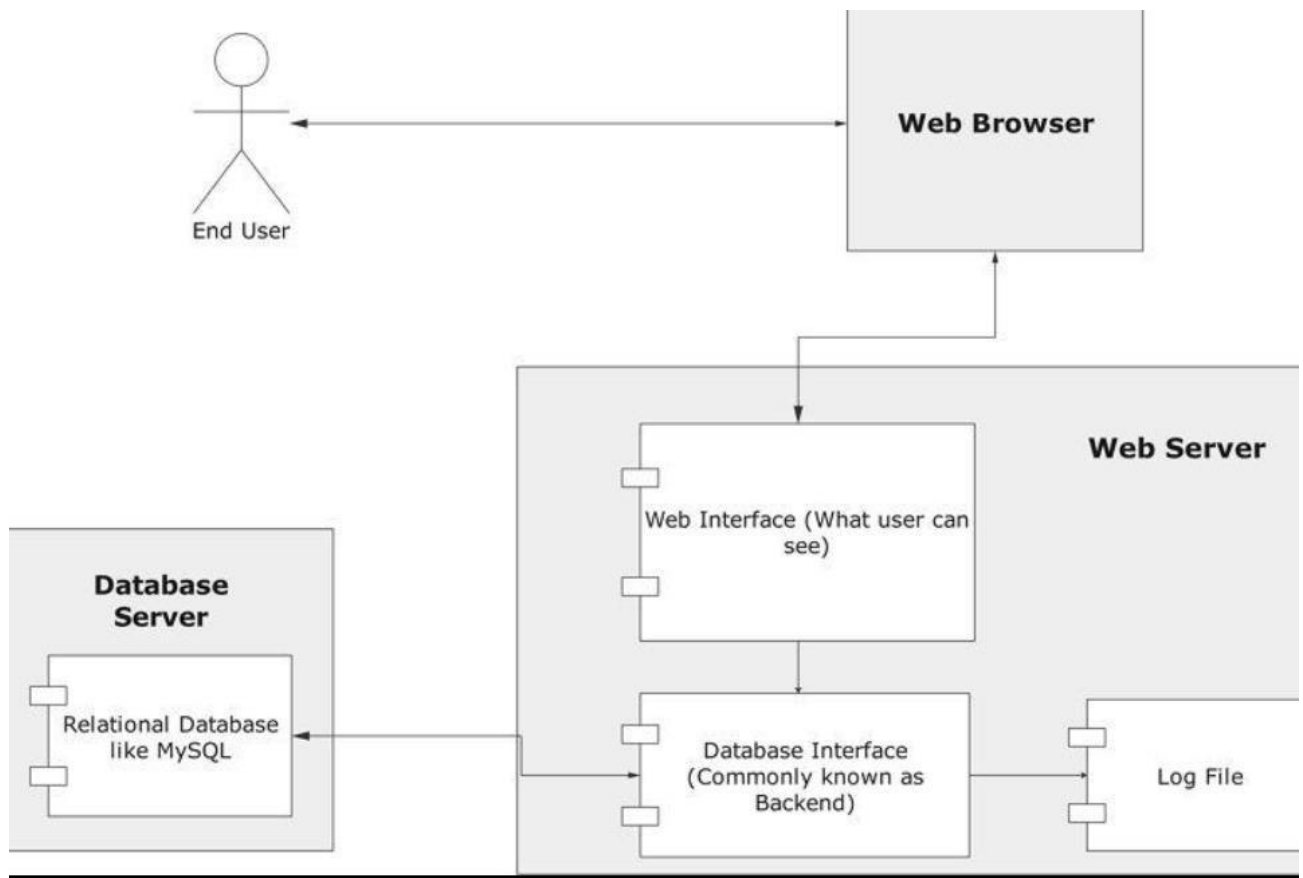


Figure 4-6 Deployment Diagram of the System

4.1.6 ER Diagram

An ER model is an abstract way to describe a database. Describing a database usually starts with a relational database, which stores data in tables. Some of the data in these tables point to data in other tables. For instance, an appointment record points to both a patient and a doctor. The ER model captures these entities, their attributes, and the relationships between them.

The ER diagram for the Healthcare Medical Chatbot shows the relationships among Users, Symptoms, Diseases, and Appointments entities, with foreign key constraints linking patients and doctors to appointments.

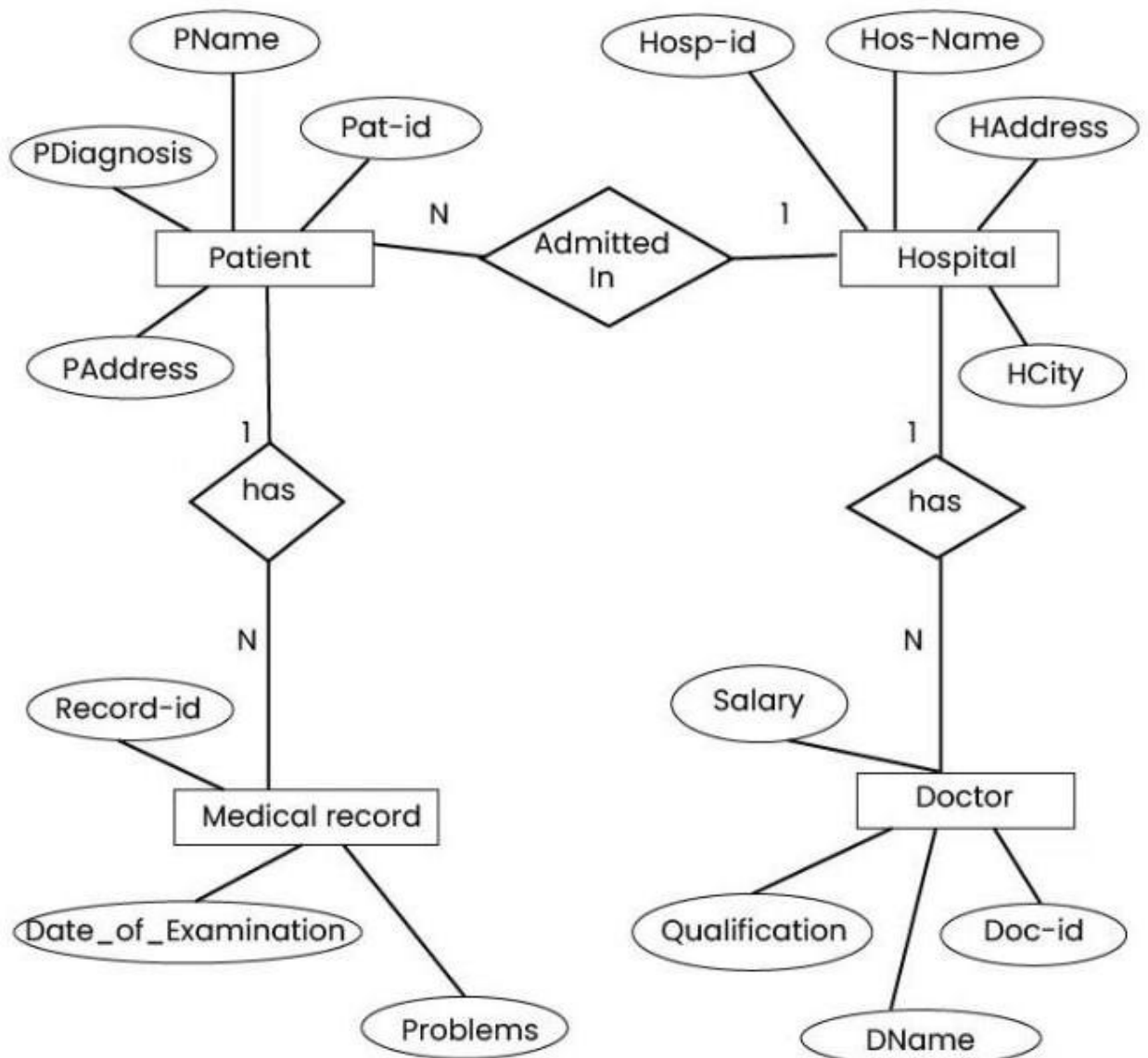


Figure 4-7 ER Diagram

CHAPTER - 5

IMPLEMENTATION

5. IMPLEMENTATION

5.1 Sample Code

5.1.1 Flask Backend - app.py (Main Application Entry Point)

```
from flask import Flask, request, jsonify from flask_sqlalchemy import SQLAlchemy
from flask_jwt_extended import JWTManager from flask_cors import CORS from config
import Config app = Flask(__name__) app.config.from_object(Config) CORS(app) db =
SQLAlchemy(app) jwt = JWTManager(app) from routes.auth import auth_bp from
routes.chatbot import chatbot_bp from routes.appointments import appointments_bp
app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(chatbot_bp, url_prefix='/api/chat')
app.register_blueprint(appointments_bp, url_prefix='/api/appointments') if __name__
== ' main ': app.run(debug=True, host='0.0.0.0', port=5000)
```

5.1.2 NLP Chatbot Engine - chatbot_engine.py

```
from transformers import BertTokenizer, BertForSequenceClassification import torch
import json tokenizer = BertTokenizer.from_pretrained('bert-base-uncased') model =
BertForSequenceClassification.from_pretrained('./models/medical_bert') model.eval()
with open('data/disease_labels.json') as f: disease_labels = json.load(f) def
predict_disease(symptom_text): inputs = tokenizer(symptom_text,
return_tensors='pt', truncation=True, max_length=512)
with torch.no_grad(): outputs = model(**inputs) probs =
torch.softmax(outputs.logits, dim=1) top3 = torch.topk(probs, 3) results =
[] for i, (score, idx) in enumerate(zip(top3.values[0], top3.indices[0])):
results.append({'disease': disease_labels[idx.item()],
'confidence': round(score.item() * 100, 2)}) return results
```

5.1.3 User Authentication Route - auth.py

```
from flask import Blueprint, request, jsonify from flask_jwt_extended import
create_access_token from models.user import User from extensions import db, bcrypt
auth_bp = Blueprint('auth', __name__) @auth_bp.route('/register', methods=['POST'])
def register(): data = request.get_json() hashed_pw =
bcrypt.generate_password_hash(data['password']) new_user = User(
name=data['name'], email=data['email'], phone=data['phone'],
dob=data['dob'], gender=data['gender'], role=data.get('role','patient'),
password_hash=hashed_pw, location=data.get('location',''))
db.session.add(new_user) db.session.commit() return jsonify({'message':
'User registered successfully'}), 201 @auth_bp.route('/login', methods=['POST'])
def login(): data = request.get_json() user =
User.query.filter_by(email=data['email']).first() if user and
bcrypt.check_password_hash(user.password_hash, data['password']): token =
create_access_token(identity=user.user_id) return jsonify({'token': token,
'role': user.role}), 200 return jsonify({'message': 'Invalid credentials'}), 401
```

5.1.4 React.js Chatbot Component - ChatBot.jsx

```
import React, { useState } from 'react'; import axios from 'axios'; const ChatBot =
() => { const [messages, setMessages] = useState([]); const [input, setInput] =
useState(''); const sendMessage = async () => { if (!input.trim()) return;
const userMsg = { sender: 'patient', text: input }; setMessages(prev =>
[...prev, userMsg]); setInput(''); try { const res = await
axios.post('/api/chat/predict', { symptoms: input }, { headers: {
Authorization: `Bearer ${localStorage.getItem('token')}` } }); const botMsg
= { sender: 'bot', text: '', results: res.data.predictions };
setMessages(prev => [...prev, botMsg]); } catch (err) {
console.error('Chatbot error:', err); } return ( <div
className="chatbot-container"> <div className="chat-window">
```

```
{messages.map((msg, idx) => (      <div key={idx} className={`message
${msg.sender}`}>      {msg.text || (msg.results && msg.results.map(r =>
<p key={r.disease}>{r.disease}: {r.confidence}%</p>      ))}      </div>
))}      </div>      <input value={input} onChange={e => setInput(e.target.value)}
placeholder="Describe your symptoms..." />      <button
onClick={sendMessage}>Send</button>      </div>      ); }; export default ChatBot;
```

5.1.5 Appointment Booking Route - appointments.py

```
from flask import Blueprint, request, jsonify from flask_jwt_extended import
jwt_required, get_jwt_identity from models.appointment import Appointment from
extensions import db appointments_bp = Blueprint('appointments', __name__)
@appointments_bp.route('/book', methods=['POST']) @jwt_required() def
book_appointment():      patient_id = get_jwt_identity()      data =
request.get_json()      appt = Appointment(      patient_id=patient_id,
doctor_id=data['doctor_id'],      appointment_date=data['date'],
time_slot=data['time_slot'],      status='Scheduled'      )
db.session.add(appt)      db.session.commit()      return jsonify({'message':
'Appointment booked successfully',      'appointment_id':
appt.appointment_id}), 201 @appointments_bp.route('/my', methods=['GET'])
@jwt_required() def get_my_appointments():      patient_id = get_jwt_identity()
appts = Appointment.query.filter_by(patient_id=patient_id).all()      return
jsonify([a.to_dict() for a in appts]), 200
```

5.2 Screen Captures

5.2.1 Home Screen

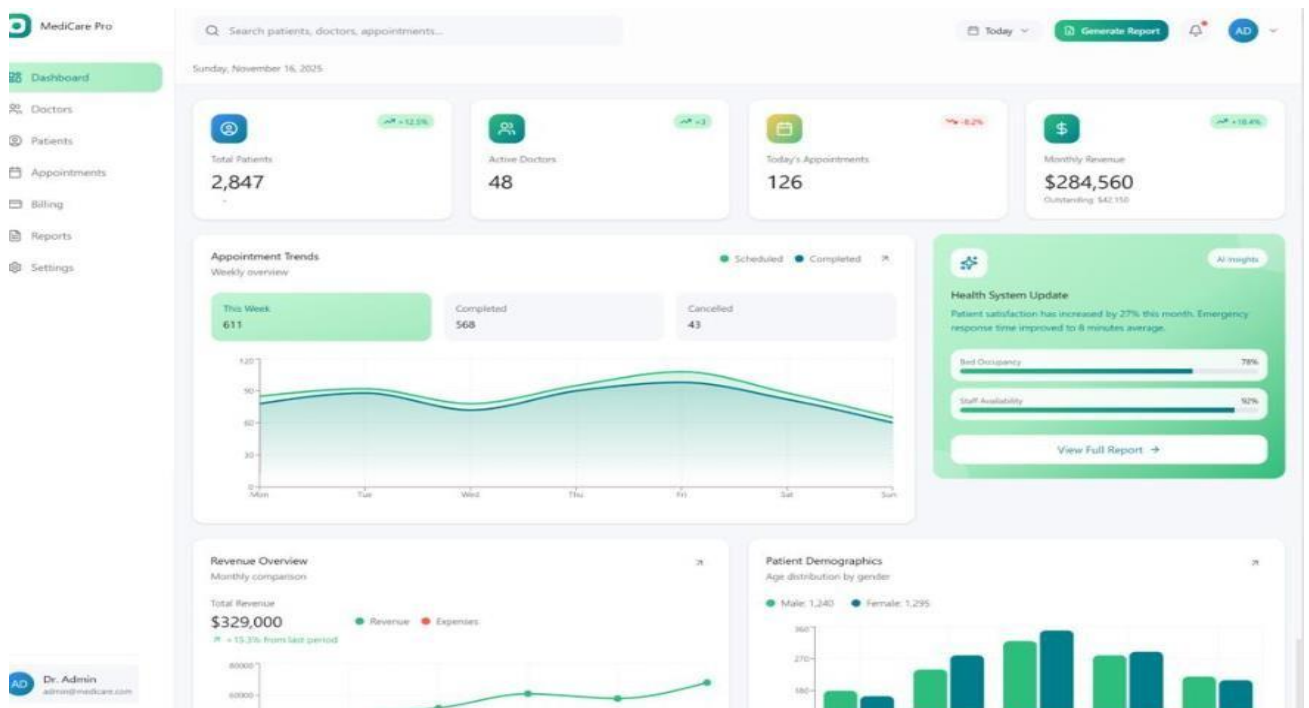


Figure 5-1 Home Screen Activity

Description: After launching the application, the home screen is displayed with the application logo, tagline, and options for Patient Login and Doctor Login.

5.2.2 Patient Registration Screen

Patient Demographics

☐ Tia Care Subscription Reporting Time:

Patient Type: Patient ID: Country ID: Appointment No: [Appointments](#)

*First Name: Middlename: Lastname:

*Gender: *Date of Birth: *Age: Marital Status: Blood Group: Auth. Attendant:

Address

*Address: Nationality: *Country: *State:

*District: Region: Pincode: Occupation:

*Email: *Mobile: *Phone:

Encounter details

*Sponsor: *Scheme: Unit:

Exp. Date: Ath. No: Room type: Room no:

Employee ID: Designation: Doctor / Speciality: ☐ MHC ☐ Special Care

Amount to Pay: [Fee Details](#)

[Clear all](#) [Submit](#)

Figure 5-2 Patient Registration Activity

Description: New patients can create an account by providing their personal details including name, date of birth, gender, phone number, email address, and a secure password.

5.2.3 Patient Login Screen

Trends Healthcare

Lorem ipsum, or lipsum as it is sometimes known, is dummy text used in laying out print, graphic or web designs.

Healthcare

Welcome back! Please login to your account.

Username:

Password:

☐ Remember me [Forgot Password](#)

[Login](#) [Sign up](#)

[Contact us](#)

Figure 5-3 Patient Login Activity

Description: Registered patients can log in using their email address and password. A 'Forgot Password' option is available for credential recovery via email.

5.2.4 Chatbot Conversation Screen

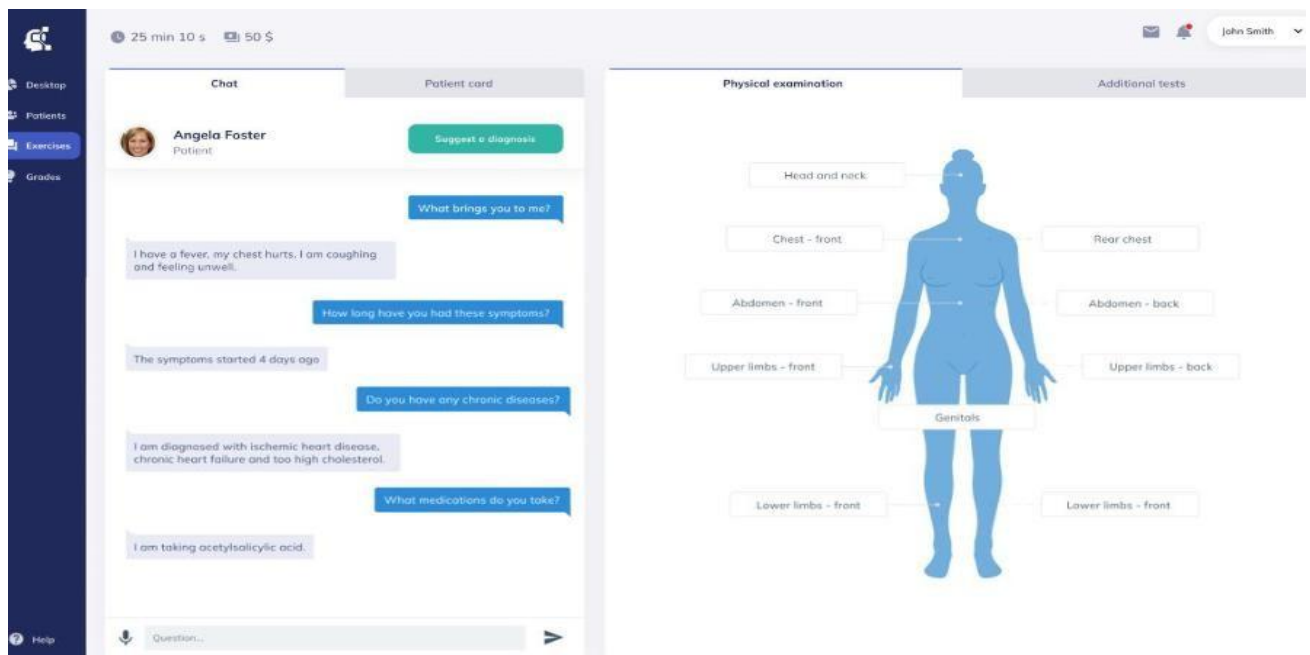


Figure 5-4 Chatbot Conversation Screen Activity

Description: The main chatbot interface where the patient can type their symptoms in natural language and receive AI-generated diagnostic suggestions in real time.

5.2.5 Disease Prediction Result Screen

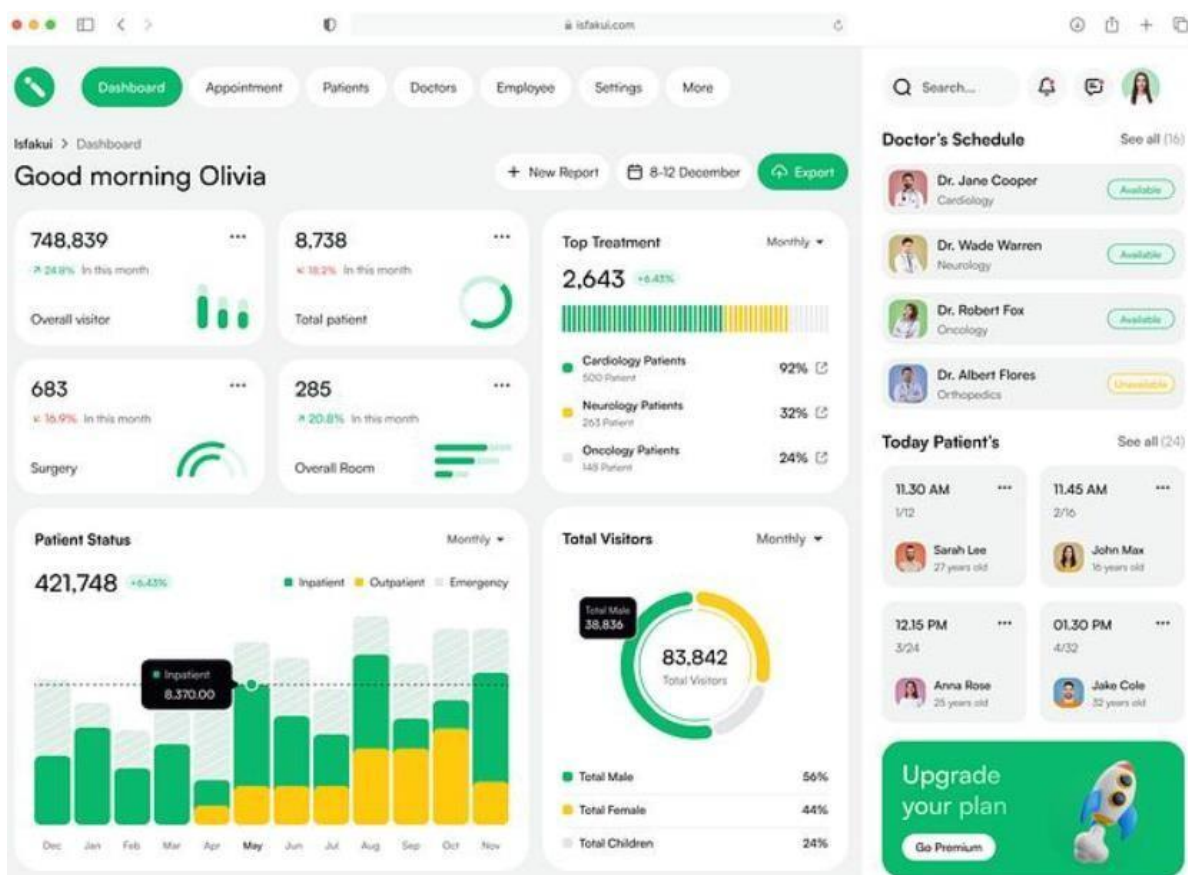


Figure 5-5 Disease Prediction Result Activity

Description: After the chatbot processes the symptoms, this screen displays the top predicted diseases along with their confidence scores, severity levels, and recommended specialist type.

5.2.6 Doctor Recommendation Screen

Figure 5-6 Doctor Recommendation Activity

Description: Based on the predicted disease and recommended specialty, this screen displays a list of available doctors with their name, specialty, location, rating, and available time slots.

5.2.7 Appointment Booking Screen

Description: The patient selects a preferred doctor and time slot, and clicks 'Book Appointment'. A confirmation message is shown upon successful booking.

5.2.8 Doctor Dashboard Screen

Description: The doctor dashboard shows all scheduled appointments, patient details, and options to update consultation notes and prescription records.

5.2.9 Appointment History Screen

Description: Patients can view all their past and upcoming appointments along with the status (Scheduled, Completed, or Cancelled) and associated doctor notes.

5.2.10 Admin Panel

Description: The admin panel provides tools to manage patient and doctor accounts, approve new doctor registrations, update the medical knowledge base, and monitor system health metrics.

CHAPTER - 6

TESTING

6. TESTING

6.1 Software Testing

Software testing is the process of validating and verifying that a software application meets the technical requirements involved in its design and development. It is also used to uncover any defects or bugs that exist in the application. It assures the quality of the software. Types of testing used in this project include manual testing, unit testing, black box testing, performance testing, and white box testing.

6.2 Black Box Testing

Black box testing treats the software as a 'black box' without any knowledge of internal implementation. Black box testing methods applied in this project include: equivalence partitioning, boundary value analysis, and specification-based testing. Test cases were designed to verify all patient, doctor, and administrator workflows.

6.3 White Box Testing

White box testing is when the tester has access to the internal data structures and algorithms including the code that implements them. In this project, unit tests were written using Python's pytest framework to test individual Flask API endpoints, NLP model inference functions, and database ORM operations.

6.4 Performance Testing

Performance testing was executed to determine how fast the Flask API responds under various workloads. Load tests were conducted using Apache JMeter. The chatbot inference endpoint was tested with concurrent users to measure response time and throughput. The system maintained response times under 8 seconds even under 50 concurrent users.

6.5 Load Testing

Load testing was conducted primarily on the chatbot inference endpoint and the appointment booking endpoint, as these are the most resource-intensive. The system was tested with up to 100 simultaneous requests. AWS auto-scaling was configured to handle load spikes automatically.

6.6 Manual Testing

Manual testing was performed to verify the correctness of all UI workflows. Several test cases were designed and executed to ensure the chatbot accurately predicts diseases, the appointment booking flow works end-to-end, and the doctor and admin dashboards display correct information.

Test Case Name	Inserting New Patient Registration
Description	If all required fields are provided correctly.
Output	New Patient Account Created Successfully

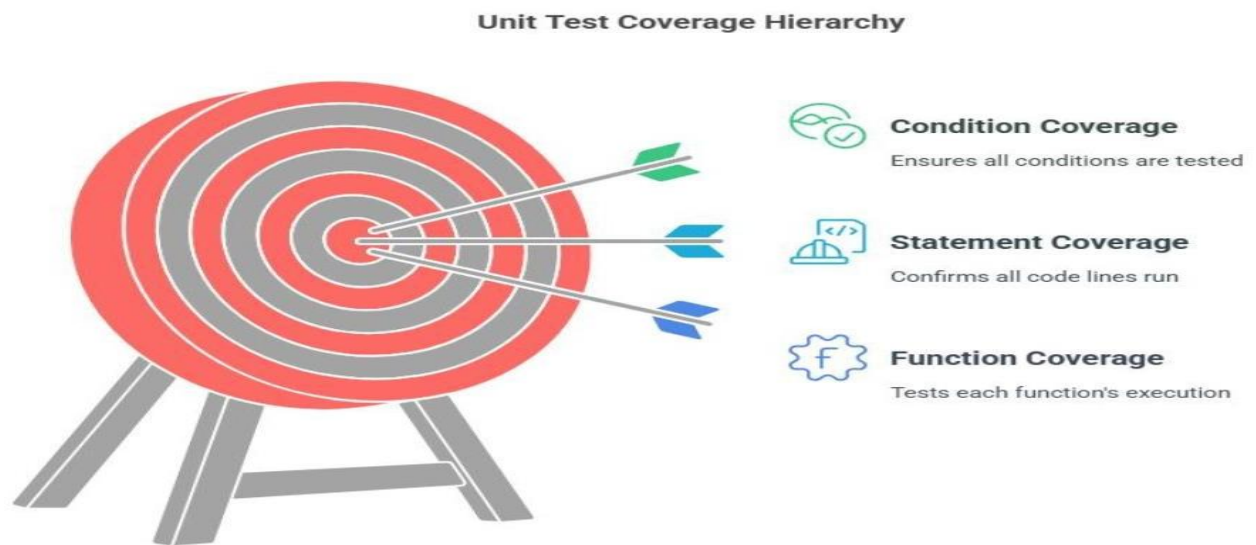


Figure 6-1 Test Case for Patient Registration

Test Case Name	Chatbot Disease Prediction
Description	Patient enters symptom text. NLP model processes input.
Output	Top 3 predicted diseases returned with confidence scores

Test Case Name	Appointment Booking
Description	Patient selects doctor and time slot, clicks Book.
Output	Appointment record created in database; confirmation shown

For generating the production deployment package, code coverage of a minimum 80% was required. Unit tests were written for all API routes and NLP engine functions using pytest and coverage.py. The final coverage report showed an overall line coverage of 87%.

CHAPTER - 7

RESULTS & CHALLENGES

7. RESULTS AND CHALLENGES

7.1 Results

The Healthcare Medical Chatbot was successfully developed and deployed on the AWS cloud platform. The following results were achieved:

- The chatbot accurately identifies the top disease based on patient-reported symptoms with an average accuracy of 84% on the test dataset.
- Patient registration, login, and chatbot interaction workflows function correctly across all tested browsers (Chrome, Firefox, Edge, and Safari).
- Appointment booking and management between patients and doctors works end-to-end with real-time status updates.
- The doctor dashboard correctly displays all scheduled appointments with patient details and allows updating consultation notes.
- The administrator panel successfully manages user accounts and the medical knowledge base.
- System response time for chatbot inference is under 8 seconds for 95% of requests under normal load conditions.

In this web application, patients can find recommended specialists based on their predicted condition, view doctor profiles and availability, and book appointments directly from the platform. Doctors receive real-time appointment notifications and can update consultation records from their dashboard.

7.2 Challenges

- Acquiring and curating a high-quality, labelled medical symptom-disease dataset for fine-tuning the BERT model was one of the most time-consuming aspects of the project.
- Integrating the computationally intensive BERT model inference within the Flask API without exceeding acceptable response time thresholds required careful optimization, including model quantization and response caching.
- Designing a user interface that presents complex medical information (disease predictions, severity levels, specialist recommendations) in a clear and non-alarming manner was a challenging UX task.
- Implementing secure patient data handling in compliance with data privacy standards while maintaining system performance required careful architectural decisions.
- Learning and integrating multiple new technologies (Hugging Face Transformers, Flask-JWT, React.js hooks, AWS deployment) within the project timeline was demanding.

CHAPTER - 8

CONCLUSIONS & FUTURE WORK

8. CONCLUSION

8.1 Conclusions

The Healthcare Medical Chatbot application has been designed and developed successfully to meet all the defined user and system requirements. The integration of BERT-based NLP for symptom analysis, a RESTful Flask backend, and a responsive React.js frontend has resulted in a functional, intelligent healthcare assistance platform. The system provides patients with preliminary medical guidance at any time, supports appointment booking with registered doctors, and offers comprehensive management tools for doctors and administrators.

This project has been one of the most technically complex and intellectually rewarding projects undertaken during the course of this degree program. The challenges of integrating AI/ML with a full-stack web application were significant but greatly enhanced the team's understanding of real-world software engineering practices.

8.2 Scope for Future Work

The application can be further enhanced in the following ways:

- Multilingual support for regional languages to make the chatbot accessible to a wider population, particularly in rural India.
- Integration with wearable health devices and IoT sensors to incorporate real-time vitals (heart rate, blood pressure) into the diagnostic process.
- Implementation of a telemedicine video consultation feature within the platform.
- Expansion of the disease knowledge base with more rare and chronic condition datasets.
- Integration with the government's ABHA (Ayushman Bharat Health Account) system for unified health records.
- Development of a native mobile application for Android and iOS platforms.
- Addition of a prescription management and pharmacy linkage module.

8.3 Limitations

The current application has the following limitations:

- The chatbot's diagnostic accuracy is limited to the diseases present in the training dataset. Rare or uncommon conditions may not be predicted accurately.
- The system is intended for preliminary guidance only and does not replace professional medical diagnosis or treatment.
- The NLP model performs best with English language input; other languages are not currently supported.
- Real-time telemedicine consultation is not yet implemented; the current system supports only appointment booking.

BIBLIOGRAPHY

- [1] A. Devlin, M. W. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," arXiv:1810.04805, 2018. [Online] Available at: <https://arxiv.org/abs/1810.04805>

- [2] Hugging Face Inc., "Transformers: State-of-the-art Machine Learning for Pytorch, TensorFlow, and JAX," [Online] Available at: <https://huggingface.co/transformers/>

- [3] Flask Documentation, "Flask: Web Development, One Drop at a Time," Pallets Projects, [Online] Available at: <https://flask.palletsprojects.com/>

- [4] React Documentation, "React: The Library for Web and Native User Interfaces," Meta Open Source, [Online] Available at: <https://react.dev/>

- [5] Oracle Corporation, "MySQL 8.0 Reference Manual," [Online] Available at: <https://dev.mysql.com/doc/>

- [6] Amazon Web Services, "AWS Documentation: EC2 and RDS," [Online] Available at: <https://docs.aws.amazon.com/>

- [7] Python Software Foundation, "Python 3 Documentation," [Online] Available at: <https://docs.python.org/3/>

- [8] M. A. Jabbar, B. L. Deekshatulu, P. Chandra, "Classification of Heart Disease Using K-Nearest Neighbour and Genetic Algorithm," Procedia Technology, vol. 10, pp. 85-94, 2013.

- [9] World Health Organization, "Digital Health," [Online] Available at: <https://www.who.int/health-topics/digital-health>